

2次元での分類

目標

- 2つの特徴量（feature）を使った分類
- 手動で直線を引く → 決定木 → ロジスティック回帰 の順で理解
- 決定境界 を可視化する

① ここがポイント

- 1次元（重さのみ）から2次元（重さ+輝度）への拡張
- まず目で見て、次にアルゴリズムで

前回の復習

1次元での分類

- 重さ だけで Salmon vs Sea bass を分類
- しきい値（threshold）を手動で設定
- TP/FP/TN/FN、Precision/Recall を学習

① ノート

問題点: 最適なしきい値は、ケースによって異なる（コスト計算などで決定）

今回のアプローチ

もう1つ特徴量を追加する

- ベルトコンベアに人感センサを搭載して、輝度（明るさ）も取れることがわかった
→ 重さや輝度のような測定値を特徴量 という
- 2つの特徴量を使えば、もっと正確に分類できるはず？

	前回	今回
特徴量	重さ（1次元）	重さ + 輝度（2次元）
境界	しきい値	？？



データの準備

新しく輝度のデータを追加した（各40匹）。

Code

```
1 # Salmon (40匹) の重さ (g)
2 salmon_weights = [2149, 1959, 2194, 2457, 1930, 1930, 2474, 2230, 1859, 2163,
3                   1861, 1860, 2073, 1426, 1483, 1831, 1696, 2094, 1728, 1576,
4                   2028, 1903, 2057, 2229, 1884, 1884, 2240, 2081, 1838, 2037,
5                   1839, 1838, 1978, 1554, 1591, 1820, 1731, 1992, 1752, 1652]
6
7 # Salmon (40匹) の輝度 (暗め: 2.1-5.9)
8 salmon_brightness = [3.5, 5.8, 4.9, 4.4, 2.6, 2.6, 2.2, 5.5, 4.4, 4.8, 2.1,
9                     5.9, 5.3, 2.8, 2.7, 2.7, 3.2, 4.1, 3.7, 3.2, 4.4, 2.6,
10                    3.2, 3.5, 3.8, 5.1, 2.8, 4.1, 4.4, 2.2, 4.4, 2.7, 2.3,
11                    5.8, 5.9, 5.2, 3.2, 2.4, 4.7, 3.8]
12
13 # Sea bass (40匹) の重さ (g)
14 seabass_weights = [1677, 1327, 1486, 1440, 1857, 1476, 1749, 2203, 1979, 1468,
15                   1496, 1737, 1099, 1341, 1748, 1563, 2181, 1414, 1286, 1333,
16                   1787, 1445, 1505, 1204, 1381, 1513, 1259, 1567, 1370, 1432,
17                   1370, 1864, 1488, 1278, 1657, 1245, 1533, 1096, 1223, 1531]
18
19 # Sea bass (40匹) の輝度 (明るめ: 5.0-9.9)
20 seabass_brightness = [5.6, 7.5, 5.2, 9.5, 6.3, 8.3, 6.6, 7.6, 7.7, 5.9, 9.8,
21                      8.9, 9.7, 9.5, 8.0, 9.6, 5.4, 6.0, 5.2, 6.6, 6.9, 6.4,
22                      9.1, 6.8, 6.4, 7.7, 5.7, 9.0, 5.4, 9.9, 8.9, 6.0, 5.0,
23                      9.1, 8.5, 8.6, 8.9, 5.4, 6.8, 5.6]
```

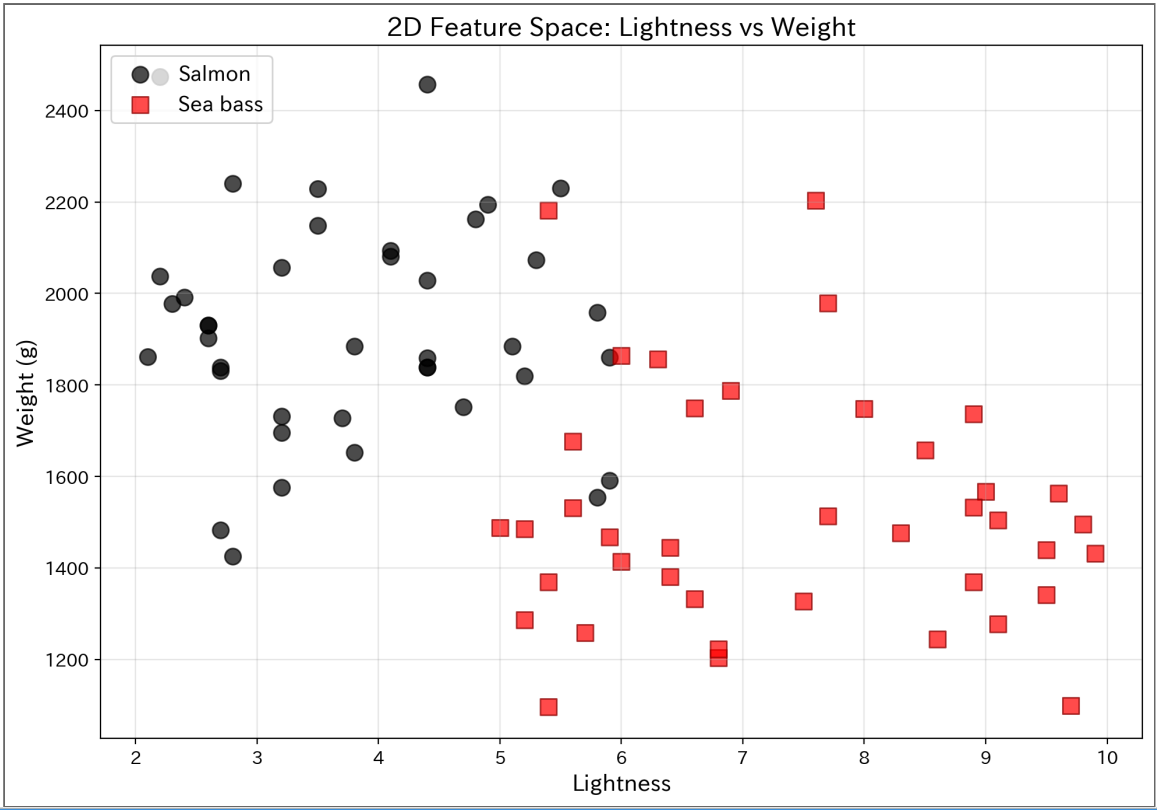
準備

必要なライブラリをインポートする。

▶ 全体を表示

目で見て境界線を想像（垂線とする）

▶ code



観察

- Salmon（黒丸）は左側（暗い）に集中
- Sea bass（赤四角）は右側（明るい）に集中
- でも **5～6あたりで少し重なっている**
- **斜めの直線で分けられそう！**

7

試行1: 目で見て直線を引いてみる

考え方

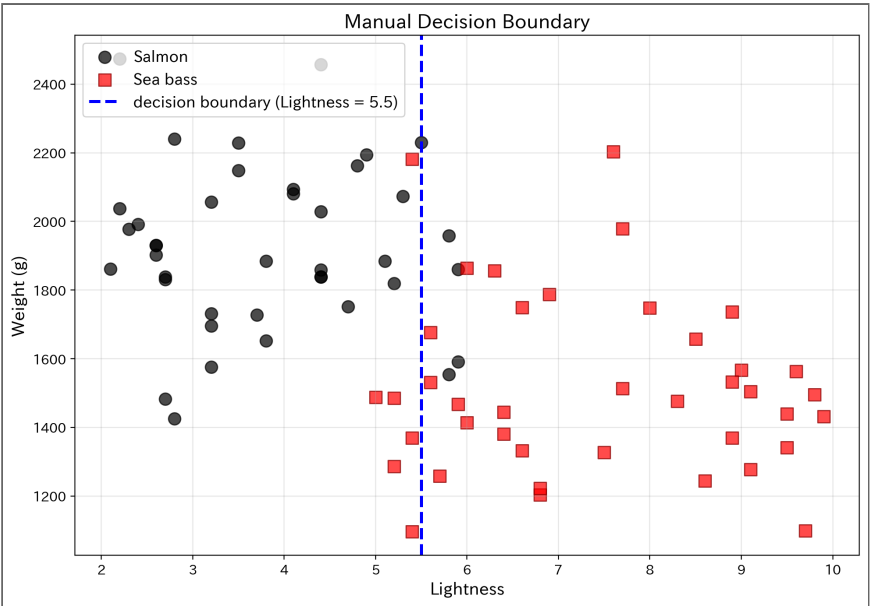
グラフを見て、**手動で直線を引いてみる。**

- どこに引けば一番うまく分けられそうか？
- 完璧には分けられない → どのくらい誤分類が出るか？

8

この辺だとうか

▶ code



💡 この直線の性能は？

輝度5.5を境界にすると、どのくらいの精度になる？ まず自分で計算してから、下の答えと突き合わせる。

Salmon を陽性 (1) とし、**輝度 < 5.5 → Salmon、>= 5.5 → Sea bass** で判定すると以下の通り。

	予測Salmon	予測Sea bass
実際Salmon	TP = 35	FN = 5
実際Sea bass	FP = 6	TN = 34

- **Accuracy** = $(35 + 34) / 80 = \mathbf{0.86}$
- **Precision (Salmon)** = $35 / (35 + 6) = \mathbf{0.85}$
- **Recall (Salmon)** = $35 / (35 + 5) = \mathbf{0.88}$
- **F1** = $2TP / (2TP + FP + FN) = 70 / 81 \approx \mathbf{0.86}$

Sea bass を陽性にする等、別の定義を使う場合は同じ手順で数え直せばよい。

試行2: 決定木 (Decision Tree)

まず：ジニ不純度とは？

決定木は「どこで分割すれば一番きれいに分かれるか」を自動で探す。

その基準がジニ不純度 (Gini impurity)。

$$\text{Gini} = 1 - (p_{\text{Salmon}}^2 + p_{\text{Sea bass}}^2)$$

- **Gini = 0**: 完全に1クラスだけ (純粋)
- **Gini = 0.5**: 50:50で混ざってる (不純)

① 例

- データ10個：Salmon 10個、Sea bass 0個 → $\text{Gini} = 1 - (1^2 + 0^2) = \mathbf{0}$
- データ10個：Salmon 5個、Sea bass 5個 → $\text{Gini} = 1 - (0.5^2 + 0.5^2) = \mathbf{0.5}$
- データ10個：Salmon 8個、Sea bass 2個 → $\text{Gini} = 1 - (0.8^2 + 0.2^2) = \mathbf{0.32}$

11

ジニ不純度を計算してみる

Code

```
1 def gini_impurity(labels):
2     """ジニ不純度を計算"""
3     if len(labels) == 0:
4         return 0
5     p_salmon = sum(labels) / len(labels) # 1=Salmon, 0=Sea bass
6     p_seabass = 1 - p_salmon
7     return 1 - (p_salmon**2 + p_seabass**2)
8
9 # 全データのジニ不純度
10 initial_gini = gini_impurity(y)
11 print(f"初期状態 (分割前) のジニ不純度: {initial_gini:.3f}")
12 print(f"  Salmon: {sum(y)}個, Sea bass: {len(y)-sum(y)}個")
```

Result

```
初期状態 (分割前) のジニ不純度: 0.500
  Salmon: 40個, Sea bass: 40個
```

12

最良の閾値をどう探す？

方法：総当たり

データの値すべてを候補として試す。

① なぜデータの値？

データとデータの**間**で分けるので、データの値を試せば十分。

例：データが [2.1, 3.5, 5.8] なら - 2.1と3.5の間で分ける = 2.1で分割と同じ結果 - 3.5と5.8の間で分ける = 3.5で分割と同じ結果

→ **データの値だけ試せばOK**

いくつかの閾値を試してみる

▶ code

Result

いくつかの閾値でのジニ不純度:

閾値	左側	右側	Gini_left	Gini_right	加重平均
3.0	13/13	27/67	0.000	0.481	0.403
4.0	22/22	18/58	0.000	0.428	0.310
5.0	32/32	8/48	0.000	0.278	0.167
6.0	40/50	0/30	0.320	0.000	0.200
7.0	40/60	0/20	0.444	0.000	0.333

→ 加重平均が最小の閾値を選ぶ

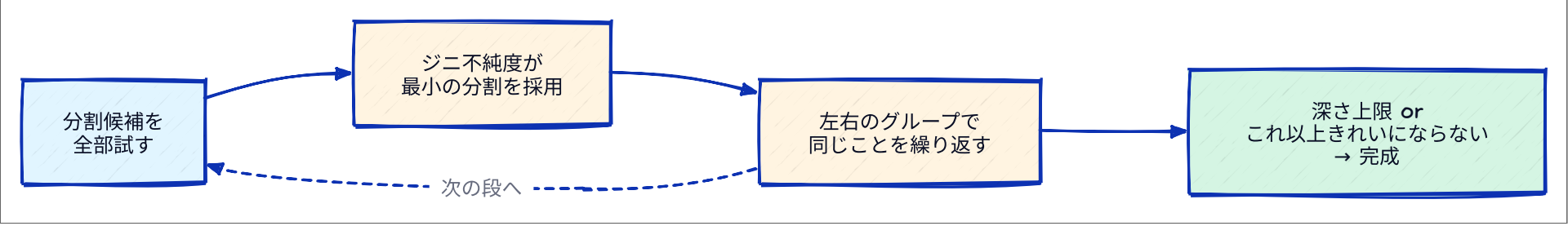
❗ 重要

実際は全てのユニーク値（80個以上）を試して、最小を見つける。

決定木のアルゴリズム（交互版）

ルール：特徴量を交互に使う

1回目は輝度、2回目は重さ、3回目は輝度...と**交互に使う**ことで、階段状の境界を作る。



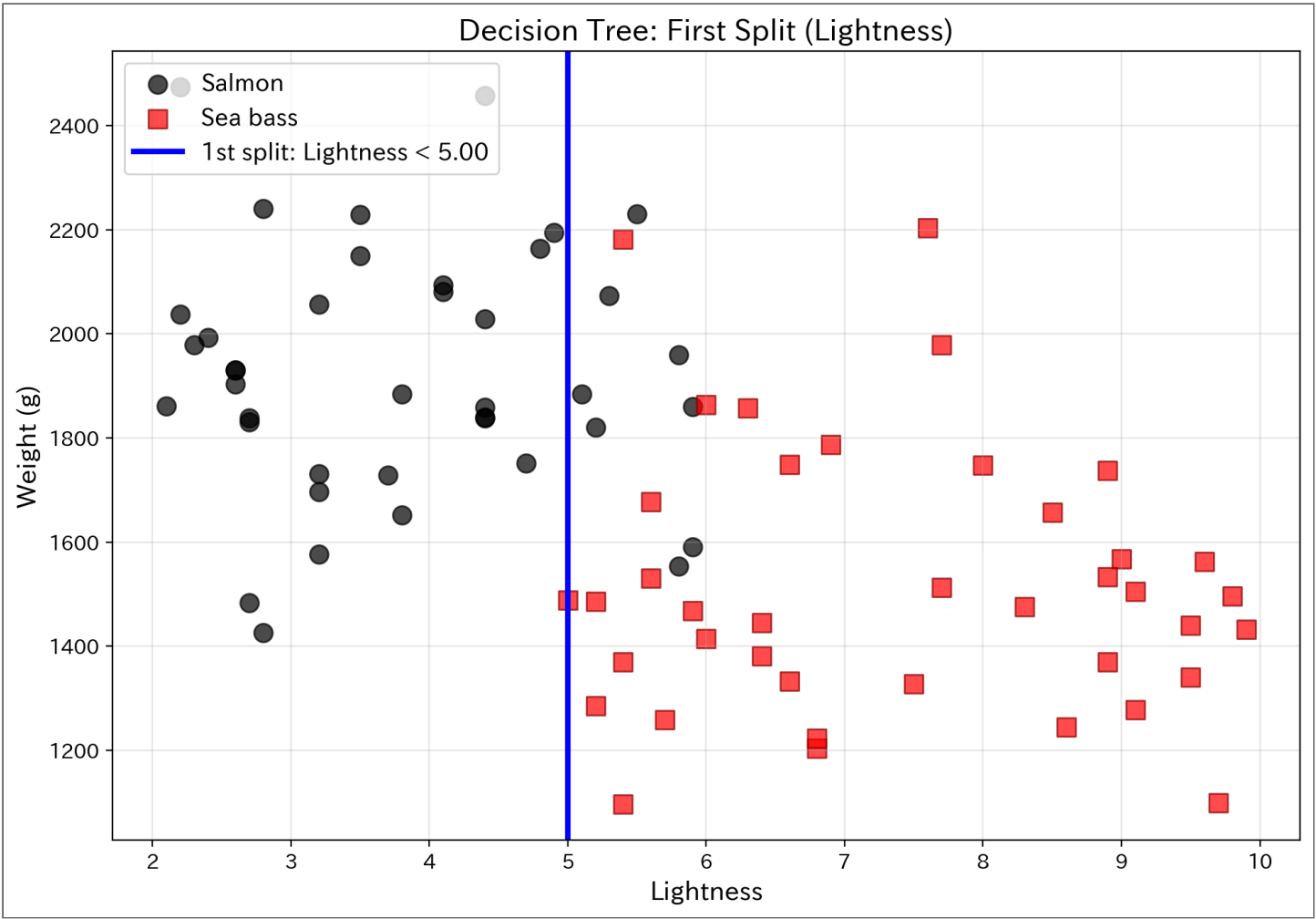
▶ code

Result

1回目の分割:
特徴量: Lightness
閾値: 5.00
ジニ不純度: 0.167
左側: 32個, 右側: 48個

15

可視化：1回目の分割



16

ステップ2: 2回目の分割（重さで分ける）

左側と右側をそれぞれ**重さで分割**する。

▶ code

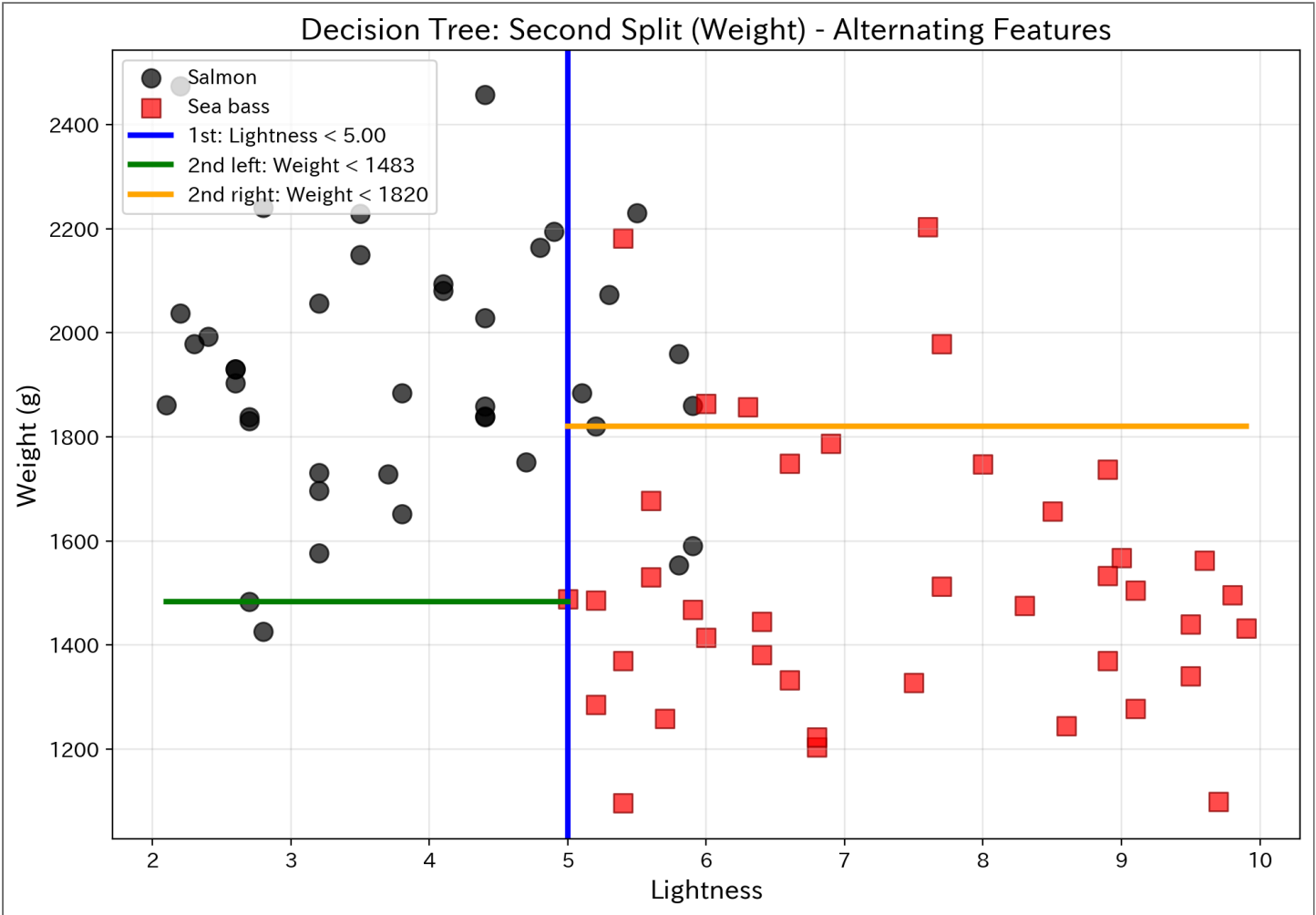
Result

2回目の分割（左側）：
特徴量: Weight
閾値: 1483
ジニ不純度: 0.000

2回目の分割（右側）：
特徴量: Weight
閾値: 1820
ジニ不純度: 0.192

17

可視化：2回目の分割



18

この境界線の性能は？

階段状の境界線で分類した結果を評価する。

▶ code

Result

交互分割決定木の性能（深さ2）：
TP=37, FP=5, FN=3, TN=35
Accuracy: 0.900
Precision: 0.881
Recall: 0.925

19

scikit-learnの決定木と比較

自作の決定木（深さ2、交互分割）とライブラリの実装（**深さ3**、最良特徴量を選択）を比較してみる。

① 違い

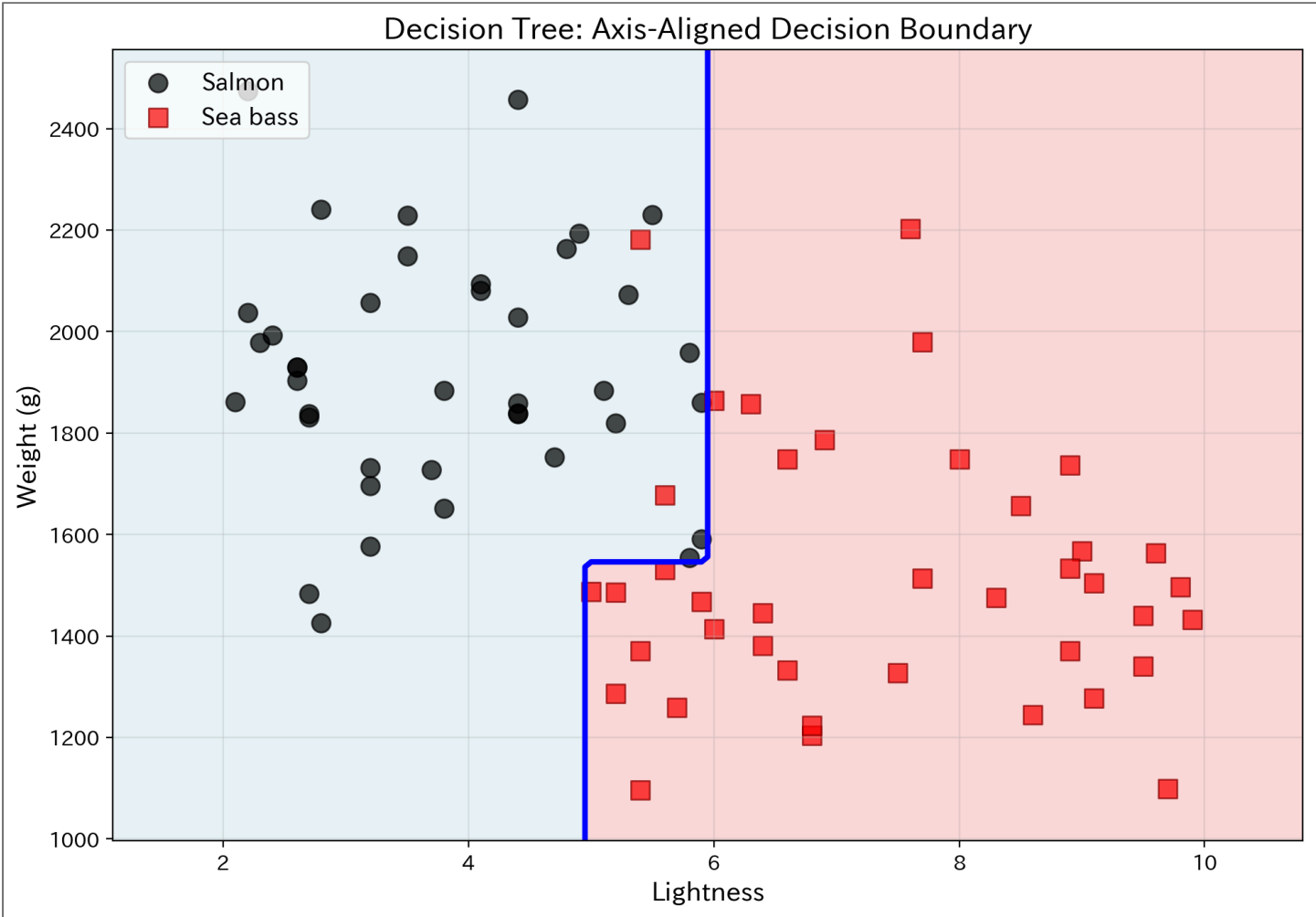
- **自作版**: 深さ2まで、輝度→重さと交互に使う
- **scikit-learn**: 深さ3まで、毎回最良の特徴量を自動選択
- → scikit-learnの方が**より細かい階段**になる

▶ code

Result

決定木の性能:
Accuracy: 0.975
Precision: 0.952
Recall: 1.000

決定境界の可視化（決定木）



① ノート

- 決定境界が**階段状**（水平・垂直の線の組み合わせ）
- if-thenルールの結果がこの形になる

試行3: ロジスティック回帰（線形分類器）

考え方

数学的に最適な直線を見つける

- 数式: $w_1 \times \text{輝度} + w_2 \times \text{重さ} + b = 0$
- この直線で2クラスを最もうまく分ける
- 斜めの直線が引ける

💡 ロジスティック回帰

線形分類器の代表的な手法。確率も計算できる。

23

ロジスティック回帰で学習

Code

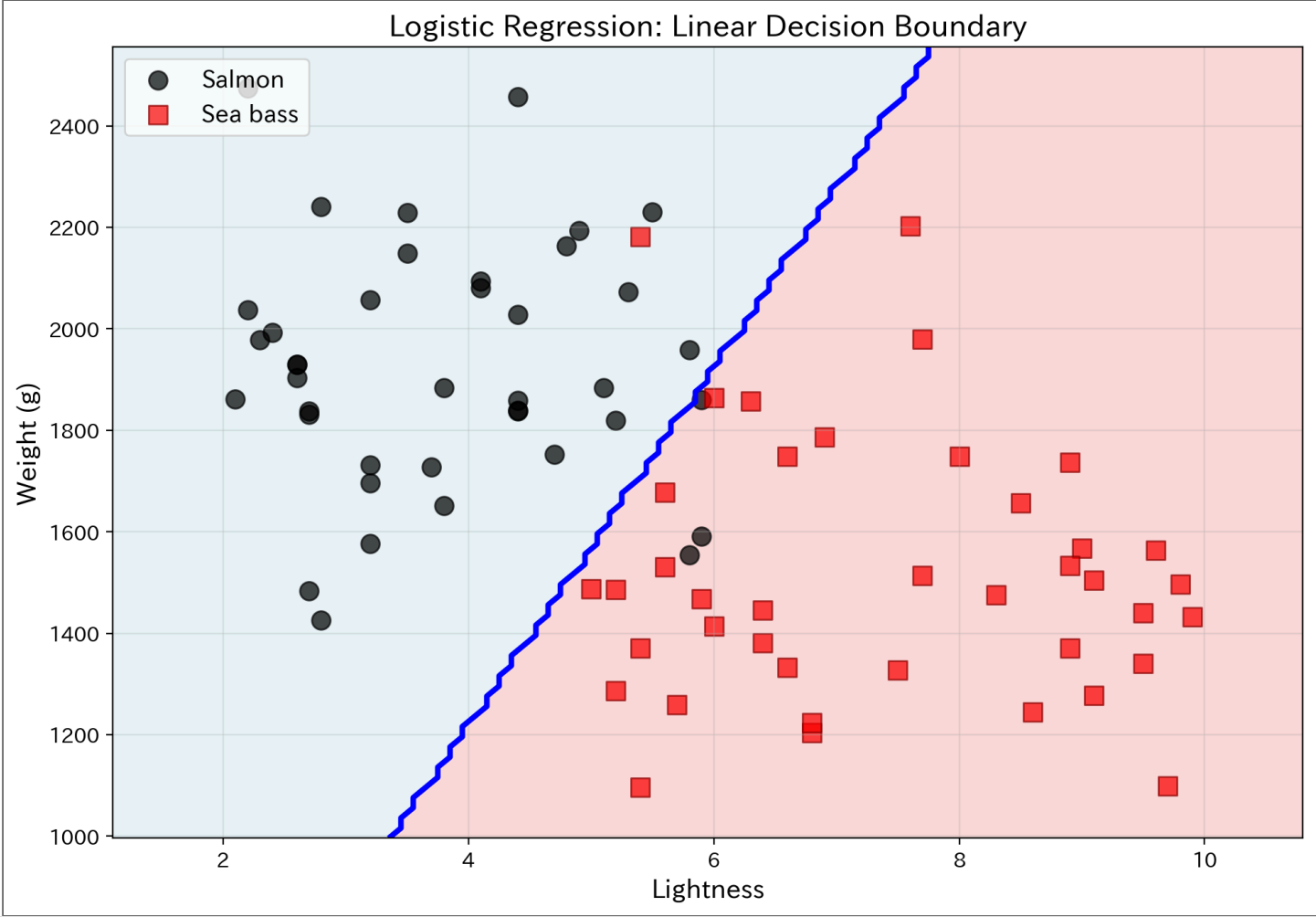
```
1 # ロジスティック回帰モデル
2 lr_model = LogisticRegression()
3 lr_model.fit(X, y)
4
5 # 予測
6 y_pred_lr = lr_model.predict(X)
7
8 # 評価
9 accuracy_lr = accuracy_score(y, y_pred_lr)
10 precision_lr = precision_score(y, y_pred_lr)
11 recall_lr = recall_score(y, y_pred_lr)
12
13 print(f"ロジスティック回帰の性能:")
14 print(f"  Accuracy:  {accuracy_lr:.3f}")
15 print(f"  Precision: {precision_lr:.3f}")
16 print(f"  Recall:    {recall_lr:.3f}")
```

Result

```
ロジスティック回帰の性能:
Accuracy:  0.950
Precision: 0.974
Recall:    0.925
```

24

決定境界の可視化（ロジスティック回帰）



① ノート

- 青い直線が**決定境界**
- 背景色: 赤＝Salmon領域、青＝Sea bass領域
- **斜めの直線1本**でシンプルに分離

25

2つのアルゴリズムの比較

決定木

- メリット:** - 解釈しやすい（if-thenルール） - 自動で最適化（ジニ不純度） - 軸に平行な境界
- デメリット:** - 過学習（**overfitting**）しやすい - 階段状の境界のみ

ロジスティック回帰

- メリット:** - 数学的に最適 - シンプル（斜めの直線） - 過学習しにくい
- デメリット:** - 線形のみ - 複雑なパターンに弱い

26

性能比較表

Result

モデル	Accuracy	Precision	Recall
決定木	0.975	0.952	1.000
ロジスティック回帰	0.950	0.974	0.925

❗ 重要

どちらが「正解」ではなく、**データと目的に応じて選ぶ**

27

決定木の深さを変えると？

- 深さ = if-then ルールの段数。深いほど複雑な境界が作れる
- 深さ 1・3・無制限で学習し、同じデータで採点してみる

▶ code

Result

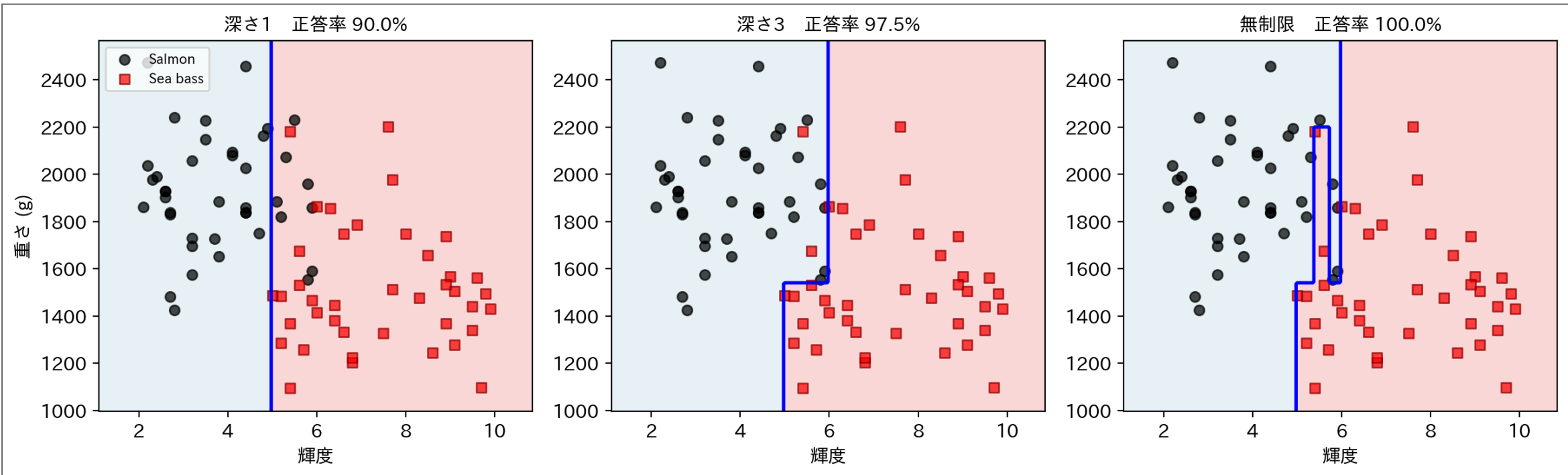
深さ1 : Accuracy = 0.900
深さ3 : Accuracy = 0.975
無制限: Accuracy = 1.000

❗ クイズ

深さ無制限なら、ついに**正答率 100%**。これは「最強のモデルが完成した」ということではない？

28

深さ無制限の境界を見てみる



- 無制限の木には、数匹を救うための**細長い回廊（飛び地）**ができている
- 実際に学習されたルールを読むと「輝度が 5.35～5.70 でも、重さが **2205g を超えるなら Salmon**」のような**異様に具体的な条項**が入っている

29

この満点、信用できるのか

- 採点に使ったのは、**学習に使ったのと同じ80匹**
- 過去問を丸暗記すれば、過去問のテストは満点になる
- あの飛び地ルールが「**次に来る魚**」にも通用するかは、この採点方法では**確かめようがない**

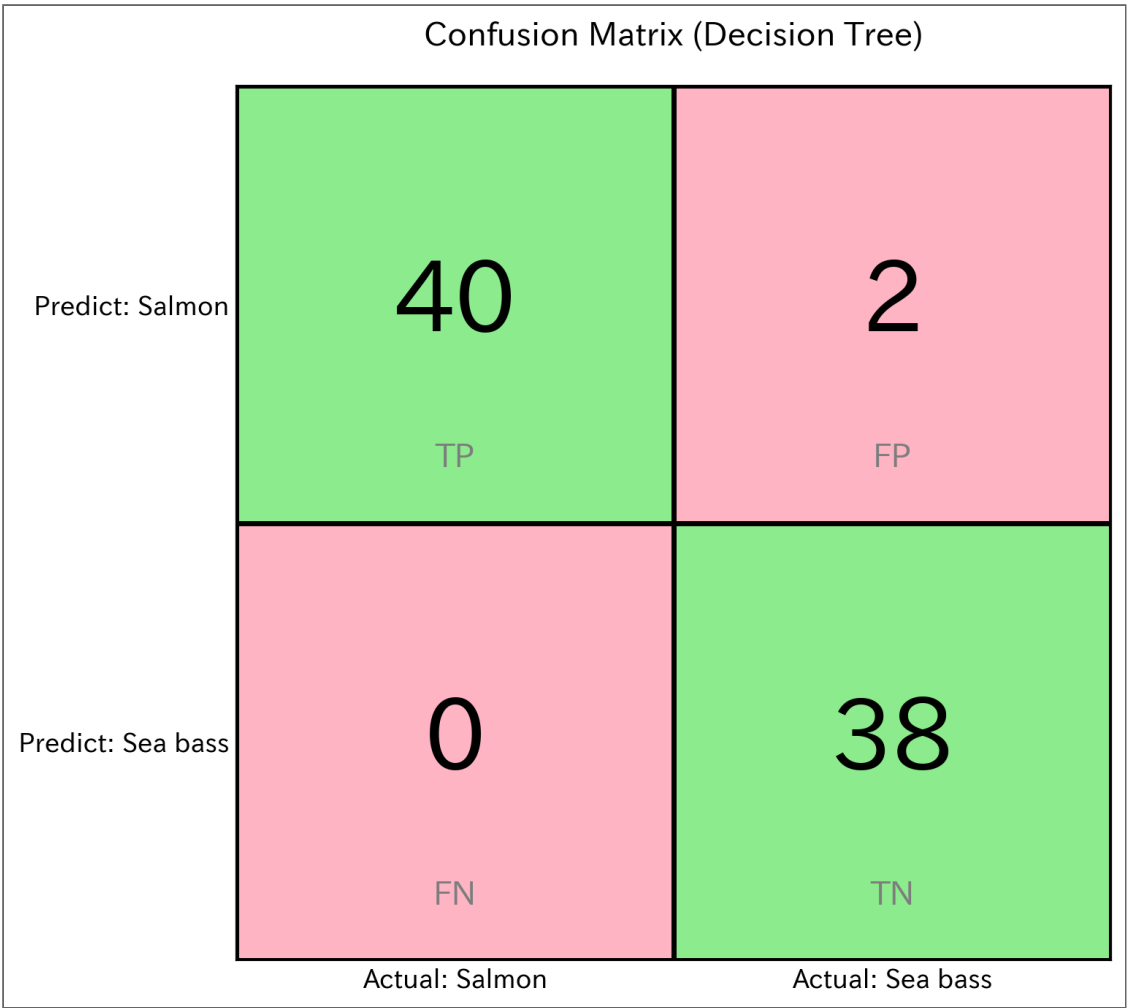
⚠ 警告

精度の数字は、**何で採点したか**とセットでないと意味を持たない。

30

混同行列で詳しく見る（決定木）

▶ code



Result

TP=40, FP=2, FN=0, TN=38

31



- 手引きの縦線 と 深さの実験 のコードを **書き換えて実行**

1	自分の縦線でスライドに勝負	<code>lightness_threshold = 5.5</code> → <code>4.8</code> など（図の誤分類を数える）
2	木を浅くする	<code>for depth in [1, 3, None]</code> → <code>[2, 3, None]</code>
3	満点が出る深さを探す	<code>[1, 3, None]</code> → <code>[3, 4, 5, 6]</code>

💡 ここで観察

- 縦線 4.8 は誤分類 **10匹**（正答率 87.5%） — スライドの 5.5（11匹、86.3%）に **1匹勝てる**。もっと勝てる線はあるか？
- 深さ3では 97.5% 止まりで、**深さ6で初めて100%** に届く — 満点までに増えた3段の if が何をしているかは、次の回で暴く

まとめ

❗ 重要なポイント

1. **2次元にすることで分類精度が向上**
 - 重さ + 輝度の組み合わせで、より正確に分類
2. **まず目で見て、次にアルゴリズム**
 - 手動の直線 → 決定木 → ロジスティック回帰
 - 段階的に理解を深める
3. **アルゴリズムによって決定境界が異なる**
 - 決定木: 階段状（軸に平行）
 - ロジスティック回帰: 斜めの直線
4. **トレードオフを理解する**
 - シンプル vs 複雑
 - 解釈しやすさ vs 性能
 - 満点に見えても信用できない場合がある（深さ無制限の決定木）

練習問題

以下を実装して、レポートで提出せよ。

1. 手動で境界線を決める

- グラフを見て、自分で境界線を決める
- その境界線でのTP, FP, FN, TNを計算

2. 決定木と ロジスティック回帰を両方実装

- 提供されたデータで学習

3. 決定木の深さを1, 2, 3, 4, 5で試す

- それぞれのAccuracy, Precision, Recallを比較
- どの深さが最適か考察

4. 考察

- 手動の境界線 vs 決定木 vs ロジスティック回帰
- どれが最も良かったか？その理由は？
- 2次元にすることで、1次元と比べてどう改善したか？

34

次回予告

- 深さ無制限の決定木は「同じデータで採点」して満点だった
- 次回、この **あやしい満点** の正体を、**新しい魚で採点する** ことで暴く
- 正体には名前がある — **過学習 (overfitting)**

35